Báo cáo thực hành dịch ngược game Android SandSort

Bài thực hành khảo sát cấu trúc APK Unity, xuất tài nguyên hình ảnh và phân tích hai luồng xử lý coin trong thư viện IL2CPP ARM64. Mục tiêu là xác định đúng lớp, phương thức, vị trí mã thực thi và đường đi của giá trị; không thực hiện sửa APK hoặc thay đổi số dư trong game.

Kết quả phân tích tĩnh cho thấy PlayerResourceManager.Start nạp coin từ PlayerPrefs với khóa Coin_SandSort, dùng 100 khi chưa tồn tại khóa. Trong nhánh nhận thưởng của UIWinGame.NextLevel, phần thực thi MoveNext gọi StateCoin với state = true và coin = 100. StateCoin cập nhật field _coin rồi lưu số dư mới. Một tài nguyên Texture2D hình chai cũng đã được xuất bằng UABEA.

1 Môi trường và dữ liệu thực hành

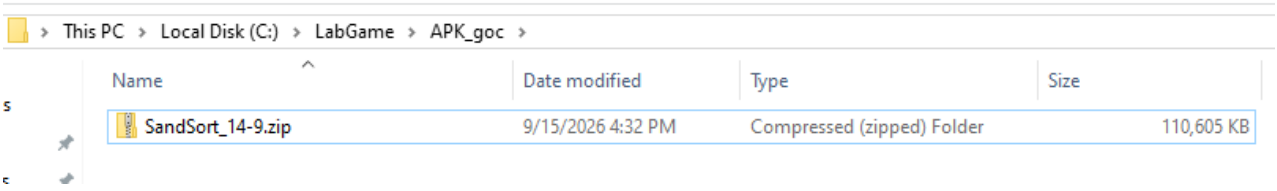
Thực hành được thực hiện trong Windows chạy trên VMware Workstation, với máy ảo có tên thuchanhdichnguoc1. Thư mục làm việc là C:\LabGame. Gói mẫu đang lưu trong APK_goc có tên SandSort_14-9.zip; dữ liệu game giải nén được đặt trong APK_giai_nen. Kiến trúc dùng xuyên suốt phần dịch ngược là ARM64.

Thành phần	Thông tin và trạng thái
Gói mẫu game	SandSort_14-9.zip, lưu tại C:\LabGame\APK_goc. Tên và đuôi file được xác nhận bằng ảnh File Explorer.
Thư viện được chọn	C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so; ELF AARCH64, little-endian, địa chỉ 64 bit.
UABEA	Gói uabea-windows; chạy UABEANext4.Desktop.exe thành công. Đã xem Preview và xuất Texture2D ở phần thực hành.
Il2CppDumper	Gói v6.7.46; ứng dụng khởi chạy đến hộp chọn file đầu vào. Lần phân tích trước đã tạo dump.cs, DummyDll và script.json.
dnSpy	v6.1.8, 64 bit .NET; cửa sổ Assembly Explorer mở được. Đã tra cứu DummyDll của game.
Ghidra	Gói v12.1.2; ghidraRun.bat mở được cửa sổ quản lý project. Đã phân tích thư viện và chạy script gắn tên thành công.

Các ảnh ở mục 1.1 đến 1.3 chứng minh vị trí dữ liệu và trạng thái khởi chạy công cụ. Kiến trúc cùng mã nhận diện libil2cpp.so được đối chiếu ở mục 1.4 và phần import Ghidra. Các công cụ được giữ trong thư mục giải nén cùng thư viện và file cấu hình đi kèm.

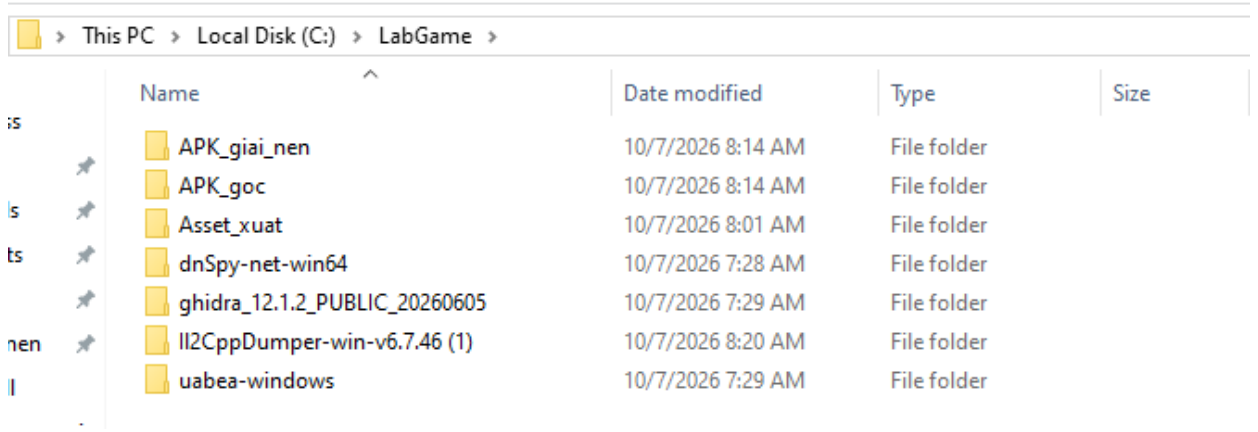
1.1 Gói mẫu và thư mục làm việc

File Explorer hiển thị đầy đủ tên SandSort_14-9.zip tại C:\LabGame\APK_goc, loại Compressed (zipped) Folder, dung lượng hiển thị 110 605 KB. Đây là tên gói đầu vào thực tế được ghi nhận. Chưa xác minh gói ZIP là APK được đổi đuôi hay gói chứa APK bên trong; versionName và versionCode của ứng dụng cũng chưa được đọc. Vì vậy, không dùng phần 14-9 trong tên file để suy ra phiên bản Android của game.



Hình 1. Tên gói mẫu SandSort_14-9.zip trong thư mục APK_goc

Thư mục C:\LabGame tập hợp APK_goc, APK_giai_nen, Asset_xuat và các thư mục công cụ. APK_goc giữ gói mẫu; APK_giai_nen chứa dữ liệu phân tích; Asset_xuat chứa tài nguyên đã xuất. Các đầu ra dump hiện nằm trong thư mục Il2CppDumper-win-v6.7.46 (1), đúng vị trí đã được dùng để chạy script Ghidra.



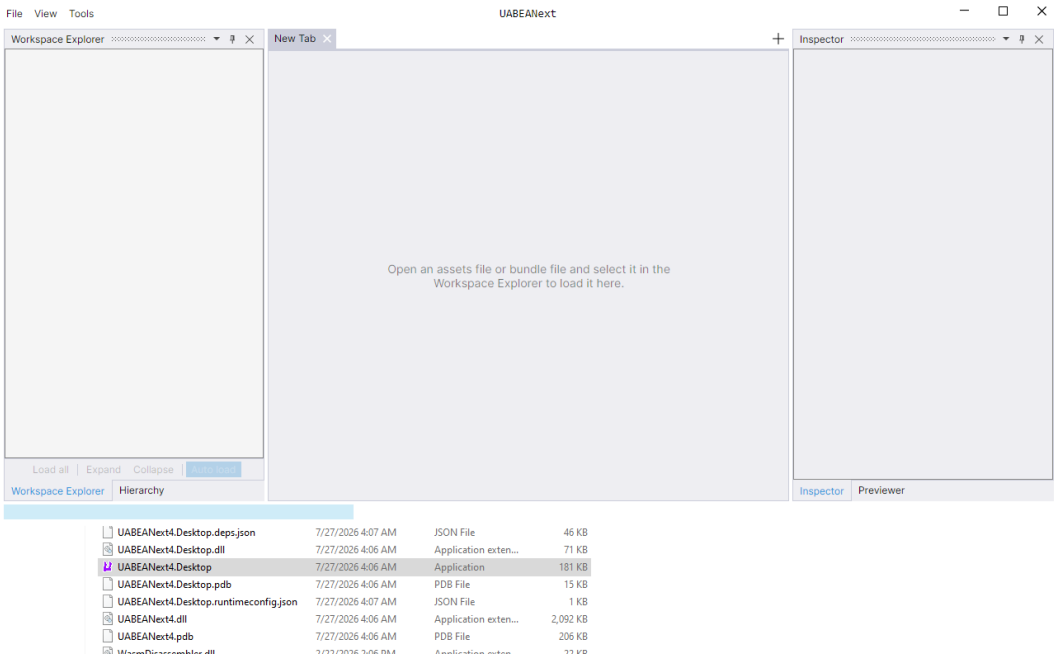
Hình 2. Thư mục làm việc LabGame với dữ liệu và các công cụ đã giải nén

Project Ghidra có tên dichnguoc và được lưu riêng tại vị trí đã chọn khi tạo project. Dữ liệu phân tích được giữ trong file project cùng thư mục dữ liệu đi kèm. Thư mục chương trình ghidra_12.1.2_PUBLIC_20260605 chứa bộ công cụ; các file dump.cs, DummyDll và script.json nằm trong thư mục Il2CppDumper được nêu trên. Các vị trí này giúp đối chiếu ảnh với kết quả thực hành.

1.2 Khởi chạy UABEA và dnSpy

UABEA

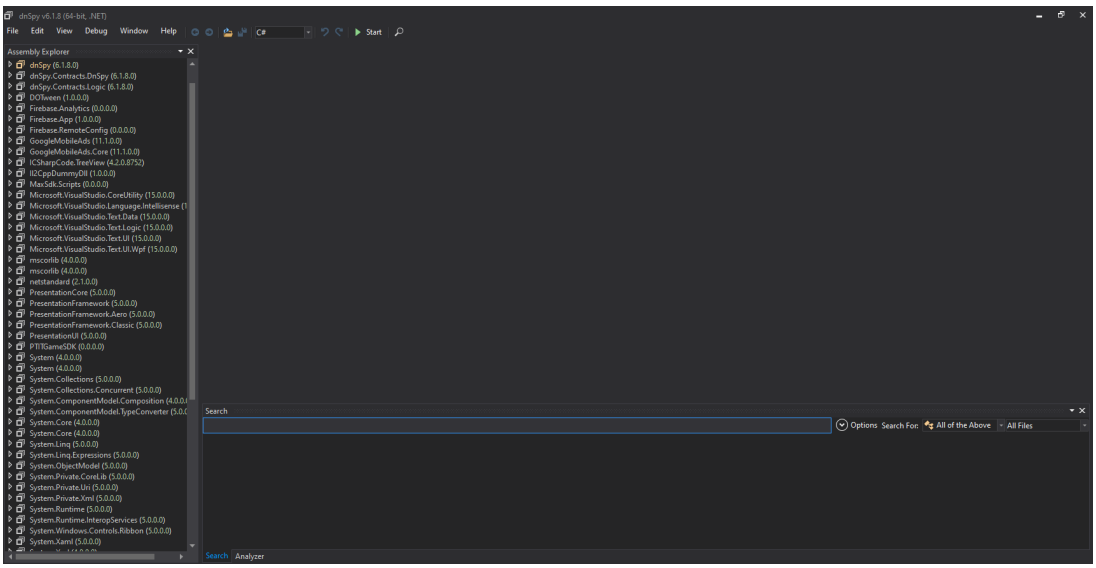
Trong C:\LabGame\uabea-windows, chạy UABEANext4.Desktop.exe. Cửa sổ UABEANext xuất hiện với Workspace Explorer, Inspector và vùng mở asset. Ảnh cho thấy ứng dụng đã khởi chạy thành công; nội dung Preview và kết quả Export được trình bày ở mục 2.



Hình 3. UABEANext mở thành công và file chạy hiện trong thư mục công cụ

dnSpy

Trong C:\LabGame\dnSpy-net-win64, chạy dnSpy.exe. Thanh tiêu đề hiển thị dnSpy v6.1.8, 64 bit .NET và bảng Assembly Explorer xuất hiện. Đây là trạng thái công cụ đã mở được; việc tra cứu lớp và phương thức trong DummyDll được đối chiếu ở mục 4.

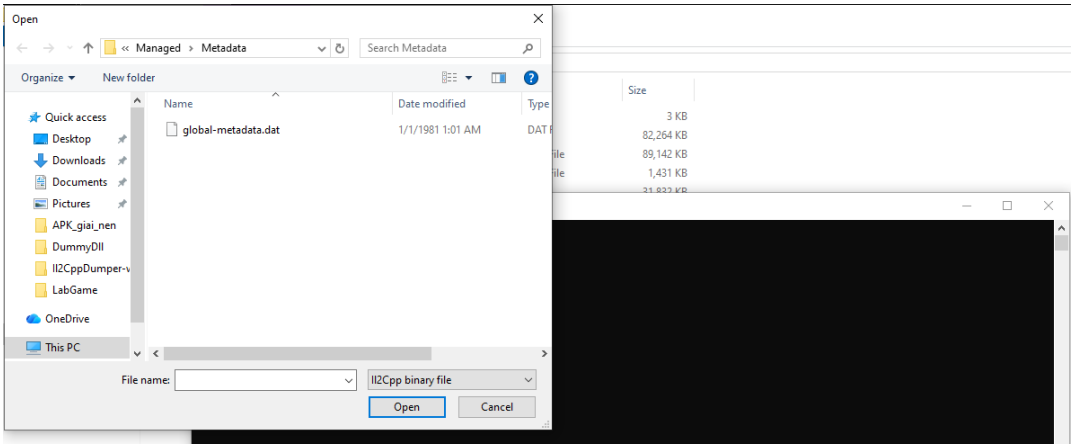


Hình 4. dnSpy v6.1.8 khởi chạy với bảng Assembly Explorer

1.3 Khởi chạy Il2CppDumper và Ghidra

Il2CppDumper

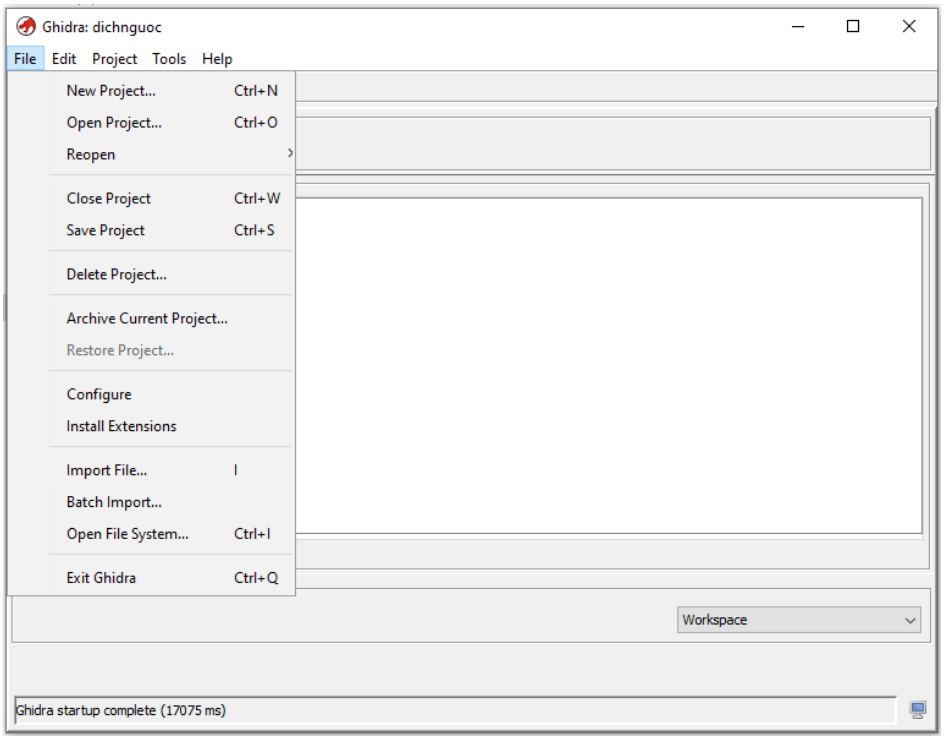
Chạy Il2CppDumper.exe trong C:\LabGame\Il2CppDumper-win-v6.7.46 (1). Console và hộp Open xuất hiện, yêu cầu Il2Cpp binary file. Ảnh chứng minh ứng dụng đã khởi chạy, chưa chọn đầu vào. Khi dump, chọn libil2cpp.so trước, rồi global-metadata.dat ở hộp tiếp theo; không dùng file metadata cho hộp chọn binary.



Hình 5. Il2CppDumper đã khởi chạy và chờ chọn file binary đầu vào

Ghidra

Chạy ghidraRun.bat trong C:\LabGame\ghidra_12.1.2_PUBLIC_20260605\ghidra_12.1.2_PUBLIC. Cửa sổ quản lý project Ghidra hiện tên dichnguoc và dòng Ghidra startup complete. Trạng thái này chứng minh bộ công cụ mở được; Listing, Decompiler và kết quả gắn tên được trình bày ở phần phân tích native.



Hình 6. Ghidra hoàn tất khởi động và mở cửa sổ quản lý project

1.4 Mã nhận diện thư viện được phân tích

Thư viện native sử dụng xuyên suốt bài là C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so. Cửa sổ Import Results Summary của Ghidra ghi nhận định dạng ELF shared object, processor AARCH64 và mã nhận diện dưới đây. Mã băm này thuộc libil2cpp.so, không phải mã băm của toàn bộ APK.

```
GNU Build ID
02f85baf0795b27bc93c81d2846ad54135610db3

MD5 của libil2cpp.so
fbfcc2616006ee6ef7bc501ec0c6b086

SHA256 của libil2cpp.so
e07d562a09889db333a1f6e1d85bb74356cc29fa92f788a8154e16ac1b5cab3f
```

Gói đầu vào đang lưu có tên SandSort_14-9.zip. Chưa xác minh versionName, versionCode hoặc tên file APK trước khi đóng gói. Mã nhận diện dưới đây thuộc chính thư viện libil2cpp.so đã import, giúp đối chiếu bản native đang phân tích; không thay thế mã băm của gói ZIP hoặc APK.

Project File Name:	libil2cpp.so
Last Modified:	Wed Oct 07 08:59:27 GMT+07:00 2026
Readonly:	false
Program Name:	libil2cpp.so
Language ID:	AARCH64:LE:64:v8A (1.6)
Compiler ID:	default
Processor:	AARCH64
Endian:	Little
Address Size:	64
Minimum Address:	00100000
Maximum Address:	_elfSectionHeaders::0000073f
# of Bytes:	86906548
# of Memory Blocks:	31
# of Instructions:	2300
# of Defined Data:	58570
# of Functions:	2180
# of Symbols:	3004
# of Data Types:	90
# of Data Type Categories:	2
Created With Ghidra Version:	12.1.2
Date Created:	Wed Oct 07 08:58:34 GMT+07:00 2026
ELF File Type:	shared object
ELF Note[GNU BuildId]:	02f85baf0795b27bc93c81d2846ad54135610db3
ELF Original Image Base:	0x0
ELF Prelinked:	false
Elf Comment[0]:	Android (7714059, based on r416183c1) clang version 12.0.8 (https://anc
Elf Comment[1]:	
Elf Comment[2]:	Linker: LLD 12.0.8 (/buildbot/src/android/llvm-r416183/out/llvm-project
Executable Format:	Executable and Linking Format (ELF)
Executable Location:	/C:/LabGame/APK_giai_nen/lib/arm64-v8a/libil2cpp.so
Executable MD5:	fbfcc2616006ee6ef7bc501ec0c6b086
Executable SHA256:	e07d562a09889db333a1f6e1d85bb74356cc29fa92f788a8154e16ac1b5cab3f
FSRL:	file:///C:/LabGame/APK_giai_nen/lib/arm64-v8a/libil2cpp.so?MD5=fbfcc261
GNU DebugLink:	libil2cpp.sym.so
Preferred Root Namespace Category:	

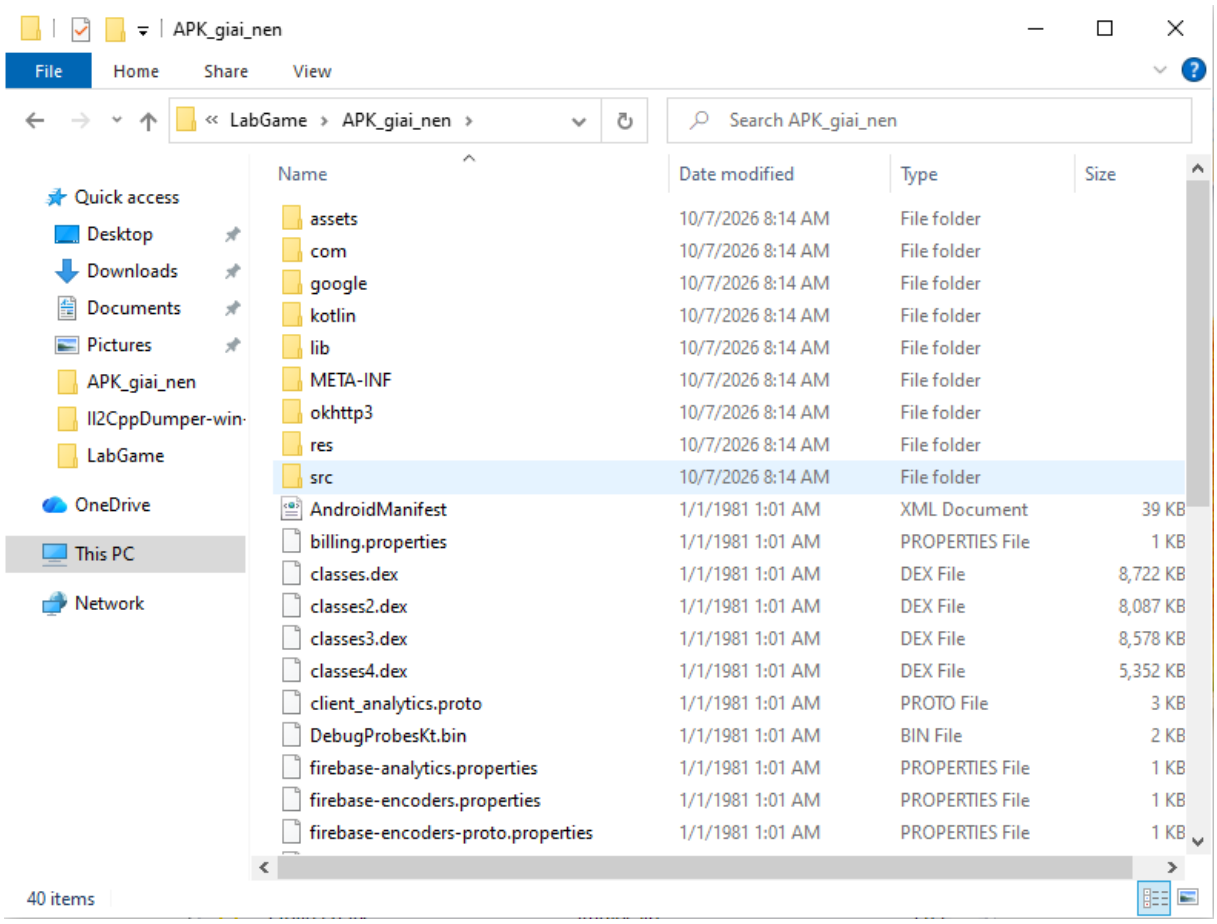
Hình 7. Thông tin kiến trúc và mã nhận diện của libil2cpp.so trong Ghidra

2 Khảo sát APK và xuất tài nguyên Unity

2.1 Cấu trúc APK sau khi giải nén

Bản sao APK được giải nén vào C:\LabGame\APK_giai_nen. Các thư mục assets, lib, res và các file classes.dex, classes2.dex cùng AndroidManifest xuất hiện trong thư mục này. Thư mục assets chứa dữ liệu Unity; lib chứa thư viện native theo kiến trúc. Các file DEX và manifest thuộc phần ứng dụng Android, không thay thế mã IL2CPP của game.

Trong UABEA, workspace data.unity3d chứa các mục sharedassets0.assets, sharedassets1.assets, sharedassets2.assets và các file dữ liệu liên quan. Dữ liệu Unity được khảo sát dưới assets/bin/Data. sharedassets0.assets là mục serialized asset bên trong workspace đang mở; không mặc định coi tên này là một file rời ngoài ổ đĩa nếu chưa kiểm tra cách đóng gói.

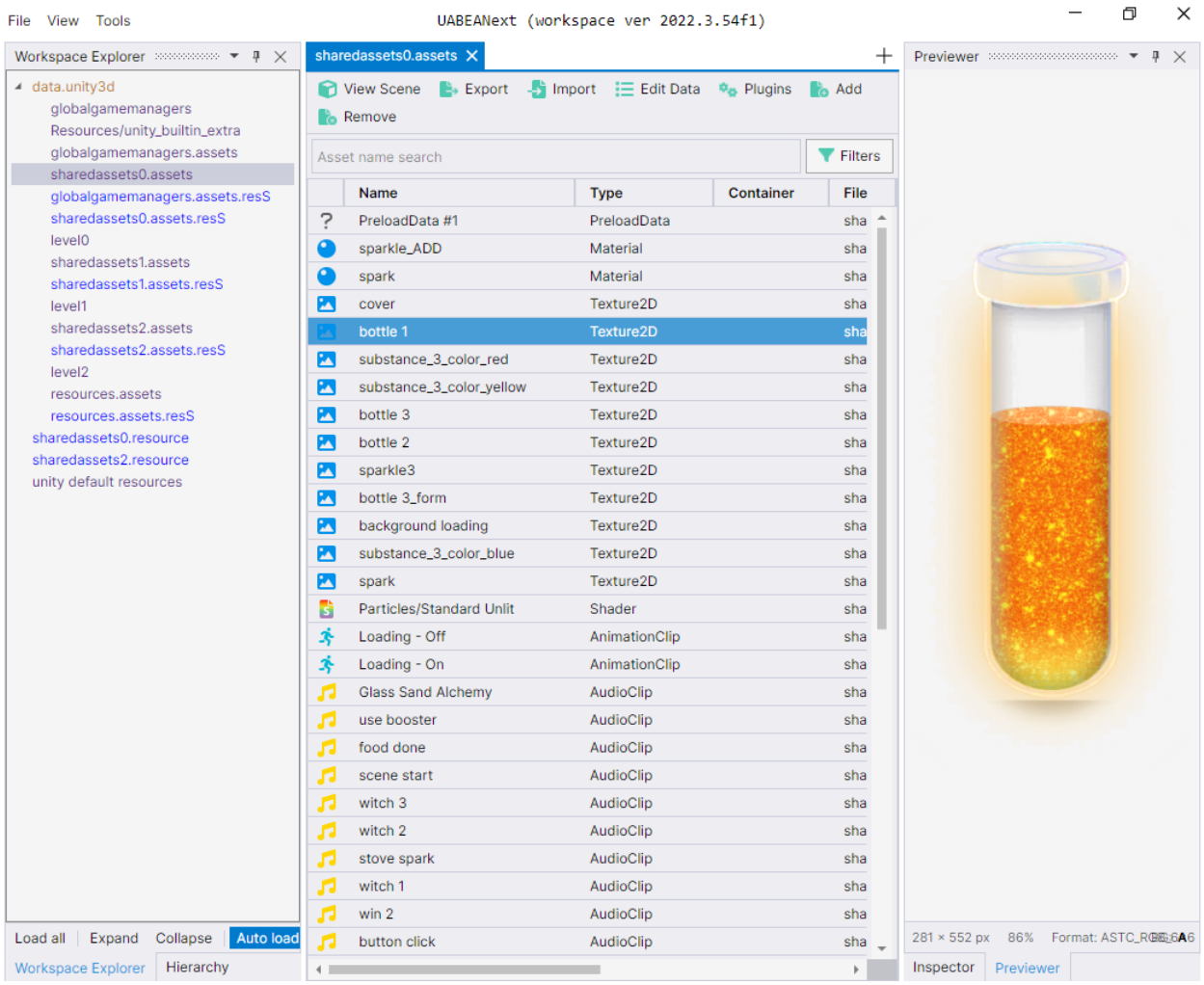


Hình 8. Cấu trúc thư mục APK_giai_nen sau khi giải nén

2.2 Xem trước tài nguyên hình ảnh

Trong workspace data.unity3d, chọn sharedassets0.assets và duyệt các đối tượng Texture2D. Asset được chọn có tên bottle 1, loại Texture2D. Khung Preview hiển thị một chai chứa vật liệu màu vàng cam; kích thước texture hiển thị là 281 × 552 pixel. Đây là tài nguyên hình ảnh, không phải phương thức xử lý coin.

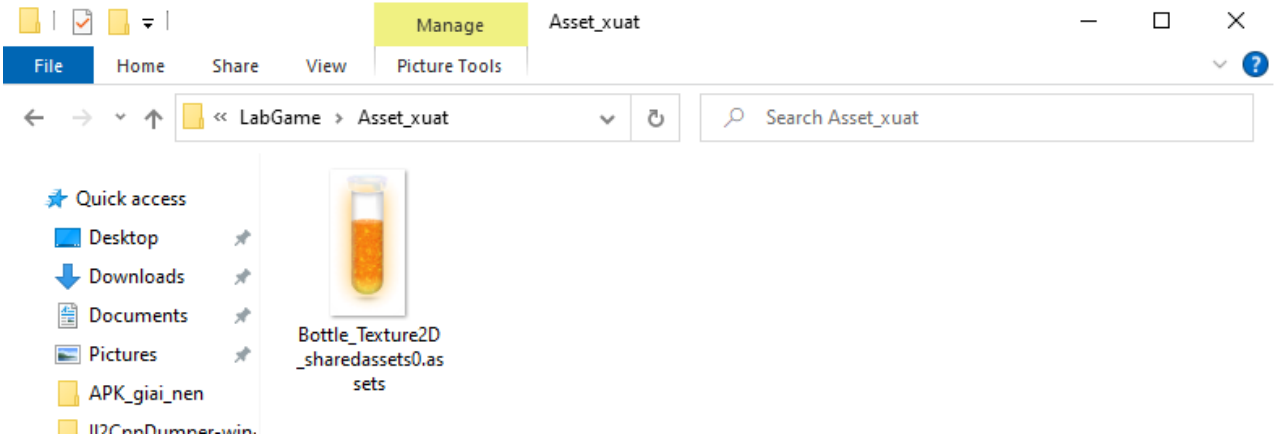
Theo hình dạng trong Preview, tài nguyên có thể phục vụ phần hiển thị chai chứa cát hoặc vật liệu màu trong nội dung game. Nhận xét này dựa trên hình ảnh tài nguyên; chưa truy vết prefab hoặc scene để khẳng định toàn bộ vị trí sử dụng.



Hình 9. Asset bottle 1 thuộc loại Texture2D trong sharedassets0.assets và ảnh Preview

2.3 Kết quả xuất asset

Sau khi xem Preview, sử dụng chức năng Export của UABEA để xuất một bản sao tài nguyên vào C:\LabGame\Asset_xuat. File Explorer hiển thị một file ảnh có thumbnail hình chai tương ứng với Preview. Tên hiển thị của file là Bottle_Texture2D_sharedassets0.assets; phần mở rộng ảnh không hiện trong ảnh chụp nên không tự khẳng định định dạng PNG hoặc định dạng khác.



Hình 10. File tài nguyên đã xuất trong thư mục Asset_xuat

Kết quả C1 gồm vị trí workspace gốc, mục sharedassets0.assets, tên bottle 1, loại Texture2D, ảnh Preview và bằng chứng file xuất. UABEA hỗ trợ nhận diện và trích xuất tài nguyên Unity; việc xác định nơi cộng coin phải được thực hiện bằng metadata và mã native ở các bước sau.

3 Tạo dữ liệu tra cứu IL2CPP

Il2CppDumper được sử dụng để đối chiếu mã máy trong libil2cpp.so với thông tin lớp và phương thức trong global-metadata.dat. Hai file đầu vào lấy từ cùng APK đã giải nén; kiến trúc ARM64 được giữ thống nhất khi import sang Ghidra.

```
File mã native
C:\LabGame\APK_giai_nen\lib\arm64-v8a\libil2cpp.so

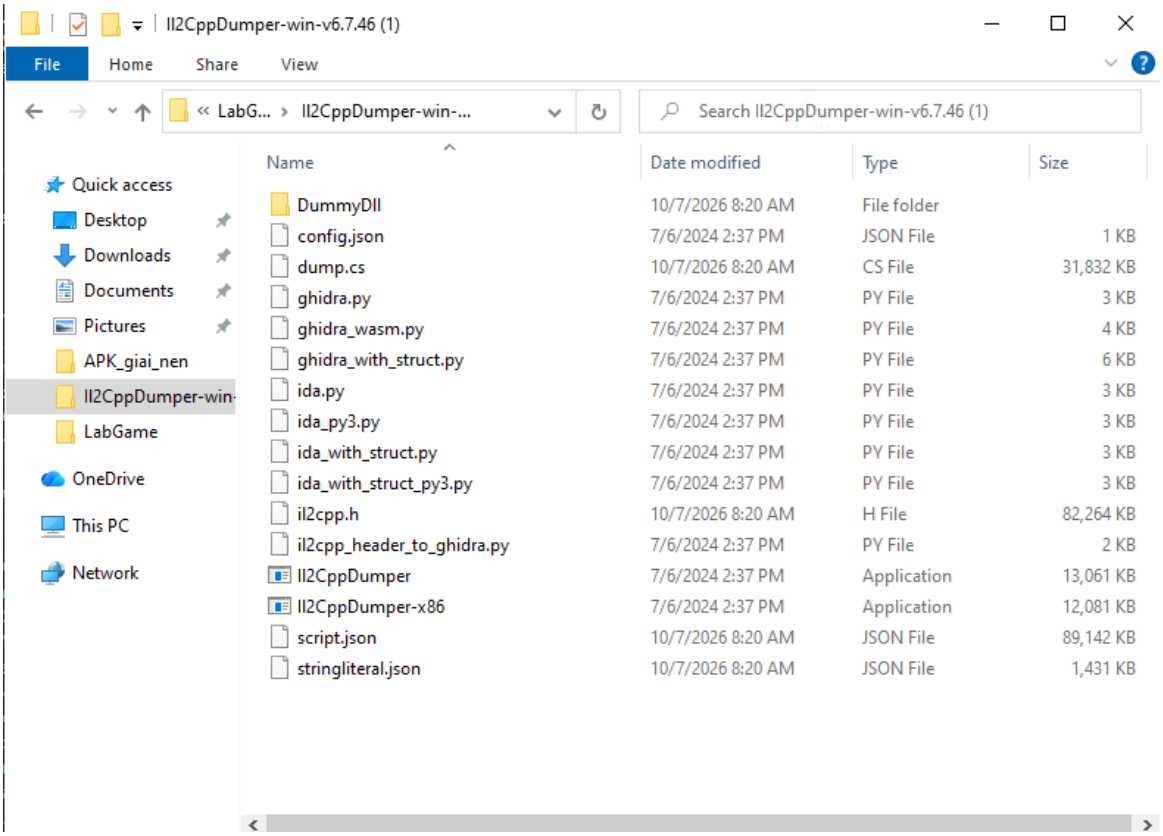
File metadata
C:\LabGame\APK_giai_nen\assets\bin\Data\Managed\Metadata\global-metadata.dat
```

Đường dẫn metadata là vị trí đầu vào theo cấu trúc IL2CPP được dùng trong bài. Các ảnh kết quả hiện thể hiện thư mục đầu ra và bản thư viện import; chưa có ảnh riêng của cả hai hộp chọn đầu vào Il2CppDumper.

3.1 Đầu ra và cách sử dụng dữ liệu dump

Thư mục kết quả Il2CppDumper-win-v6.7.46 (1) có DummyDll, dump.cs, script.json, ghidra.py, il2cpp.h và stringliteral.json. Các file này phục vụ tra cứu, gắn tên và đối chiếu địa chỉ; không phải toàn bộ mã nguồn C# gốc của game.

Đầu ra	Vai trò trong bài
dump.cs	Tra tên lớp, field, chữ ký phương thức, RVA và File Offset.
DummyDll	Mở Assembly-CSharp.dll trong dnSpy để xem cấu trúc namespace và thành viên.
script.json	Cung cấp tên và địa chỉ để script gắn nhãn trong Ghidra.
ghidra.py	Áp dụng dữ liệu script.json vào chương trình native đã import.
il2cpp.h và stringliteral.json	Dữ liệu hỗ trợ về kiểu và chuỗi; chỉ sử dụng khi cần đối chiếu thêm.



Hình 11. Các file đầu ra của Il2CppDumper trong thư mục làm việc

4 Tìm ứng viên bằng dnSpy và dump.cs

4.1 Xác định field chứa số coin

Mở DummyDll\Assembly-CSharp.dll trong dnSpy, duyệt namespace FoodSort và tìm từ khóa coin. Trong PlayerResourceManager tìm thấy private int _coin với FieldOffset 0x20. Kiểu int phù hợp với giá trị số lượng coin; vai trò của field được kiểm chứng tiếp bằng lệnh đọc và ghi trong mã native.

Kết quả tìm kiếm cũng có _coinGain của UIWinGame, nhưng field này có kiểu Transform. Vì vậy, nó được xem là ứng viên tài nguyên giao diện, không dùng làm bằng chứng trực tiếp cho biến số dư. Tên field chỉ định hướng tìm kiếm; kết luận xử lý coin phải dựa trên đường đi của giá trị.

PlayerResourceManager

```
659         [FieldOffset(Offset = "0x8")]
660         public static Action<int> OnBoosterUndoChange;
661
662         // Token: 0x04000A14 RID: 2580
663         [Token(Token = "0x4000A14")]
664         [FieldOffset(Offset = "0x10")]
665         public static Action<int> OnBoosterExtraChange;
666
667         // Token: 0x04000A15 RID: 2581
668         [Token(Token = "0x4000A15")]
669         [FieldOffset(Offset = "0x18")]
670         public static Action<int> OnBoosterShuffleChange;
671
672         // Token: 0x04000A16 RID: 2582
673         [Token(Token = "0x4000A16")]
674         [FieldOffset(Offset = "0x20")]
675         private int _coin;
676
677         // Token: 0x04000A17 RID: 2583
678         [Token(Token = "0x4000A17")]
679         [FieldOffset(Offset = "0x24")]
680         private int _numOfBoosterUndo;
681
682         // Token: 0x04000A18 RID: 2584
683         [Token(Token = "0x4000A18")]
684         [FieldOffset(Offset = "0x28")]
685         private int _numOfBoosterExtra;
686
687         // Token: 0x04000A19 RID: 2585
688         [Token(Token = "0x4000A19")]
689         [FieldOffset(Offset = "0x2C")]
690         private int _numOfBoosterShuffle;
691     }
692 }
```

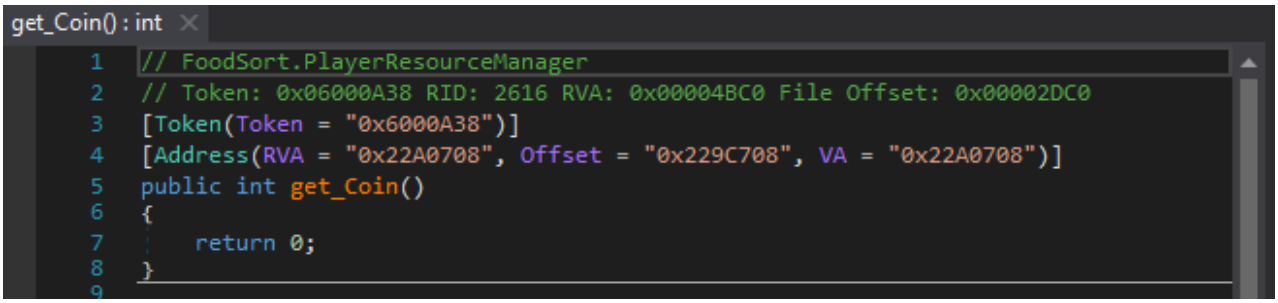
Hình 12. Field _coin kiểu int trong FoodSort.PlayerResourceManager có offset 0x20

4.2 Danh sách phương thức ứng viên và đối chiếu RVA

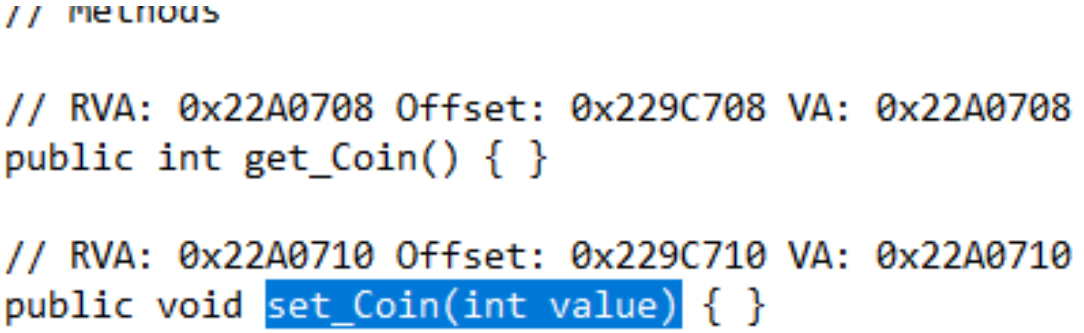
Các phương thức get_Coin, set_Coin, Awake, Start và StateCoin được chọn để kiểm tra. Trong dnSpy, địa chỉ native được lấy từ thuộc tính Address, sau đó đối chiếu dòng RVA trong dump.cs. RVA ở chú thích của DLL giả và Token metadata không được dùng thay cho địa chỉ native.

Phương thức	RVA	File Offset
get_Coin	0x22A0708	0x229C708
set_Coin	0x22A0710	0x229C710
Awake	0x22A15D0	0x229D5D0
Start	0x22A15D4	0x229D5D4
StateCoin	0x22A16F8	0x229D6F8

get_Coin trả về int, không có tham số; set_Coin nhận int value và trả về void. Thân hàm return 0 hoặc rỗng trong DummyDll là nội dung dựng phục vụ tra cứu, không chứng minh giá trị coin bằng 0 hoặc hàm không hoạt động.



Hình 13. Khai báo get_Coin và thuộc tính Address trong dnSpy



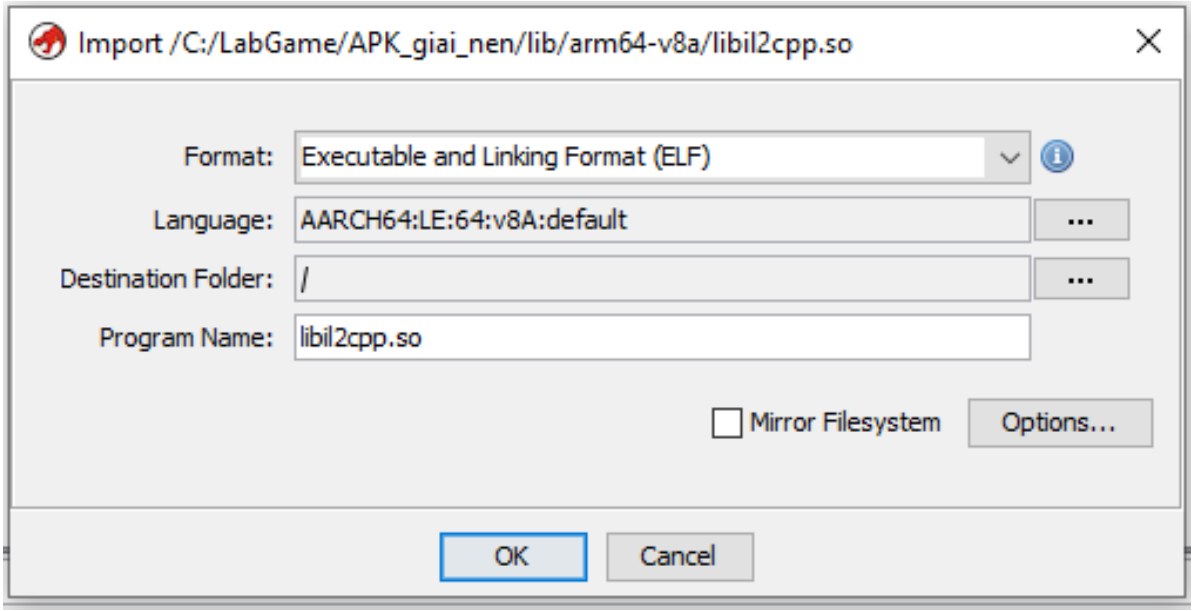
Hình 14. RVA và File Offset của get_Coin và set_Coin trong dump.cs

StateCoin có chữ ký public void StateCoin(bool state, int coin, string placement = "unknown", string groupPlacement = "unknown"). Hướng cộng hoặc trừ của state chưa được suy đoán ở bước này; được xác nhận trong Ghidra ở mục 7.

5 Import và gắn tên phương thức trong Ghidra

5.1 Import đúng thư viện ARM64

Tạo project Non-Shared tên dichnguoc và import libil2cpp.so đã dùng trong lần dump. Hộp import nhận dạng Executable and Linking Format (ELF), language AARCH64:LE:64:v8A:default. Sau khi mở CodeBrowser, thực hiện phân tích tự động để nhận diện các vùng mã, hàm và tham chiếu.



Hình 15. Import libil2cpp.so với định dạng ELF và ngôn ngữ AARCH64

5.2 Quy ước địa chỉ

Ghidra sử dụng Image Base 0x100000 trong lần import này. Ví dụ, get_Coin có RVA 0x22A0708 và được gắn nhãn tại 0x23A0708. File Offset là vị trí trong file ELF, còn địa chỉ Ghidra là vị trí trong không gian địa chỉ của chương trình đã import; hai giá trị không được tráo đổi.

```
Địa chỉ Ghidra = Image Base + RVA
0x100000 + 0x22A0708 = 0x23A0708
```

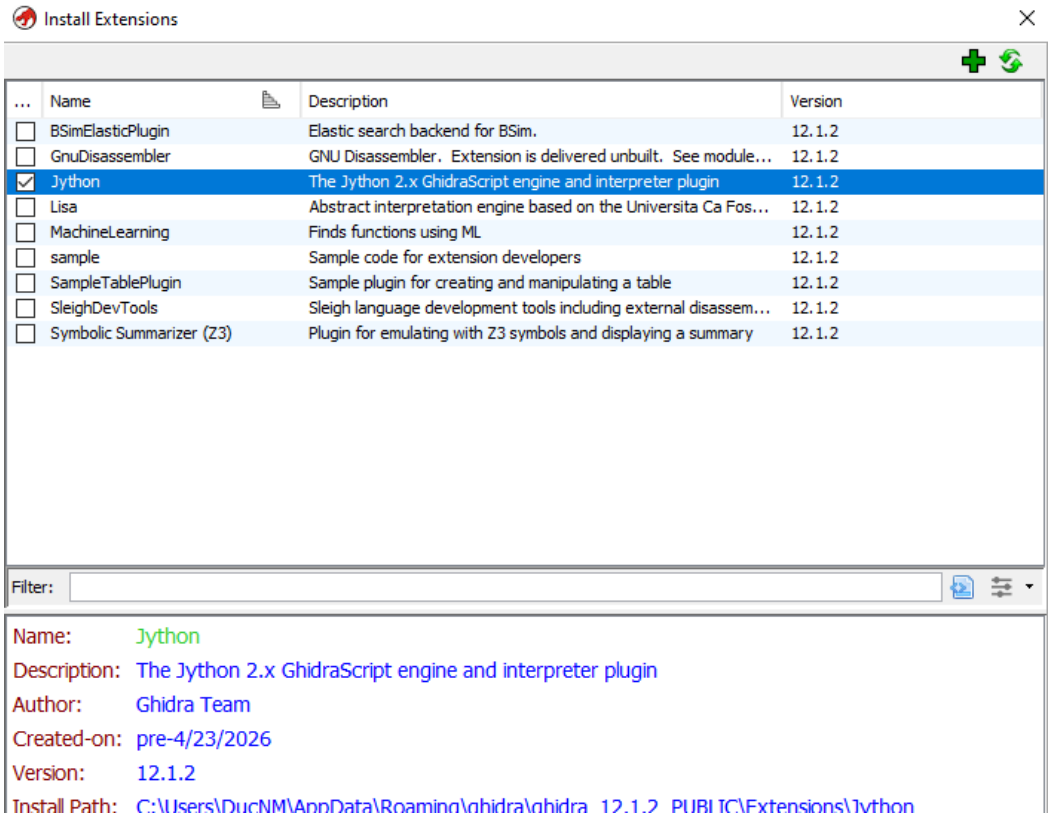
Listing cung cấp lệnh ARM64 và địa chỉ; Decompiler dựng mã giả để giải thích luồng xử lý. Mã giả được đối chiếu với khai báo dnSpy và các offset, không coi là mã C++ hoặc C# gốc.

5.3 Khắc phục môi trường chạy script và hoàn tất gắn tên

Thêm thư mục Il2CppDumper-win-v6.7.46 (1) vào Script Directories. Chọn cả thư mục, không thêm riêng file ghidra.py làm bundle. Lần chạy đầu báo Ghidra was not started with PyGhidra. Python is not available, cho thấy script chưa được thực thi bằng môi trường tương thích.

Trong cửa sổ quản lý project, mở File > Install Extensions, bật Jython rồi khởi động lại Ghidra. Do script Il2CppDumper sử dụng kiểu Python 2/Jython, thêm chỉ thị runtime vào đầu ghidra.py để Ghidra chọn đúng môi trường:

```
# -*- coding: utf-8 -*-
# @runtime Jython
import json
```



Hình 16. Extension Jython phiên bản 12.1.2 được bật trong Ghidra

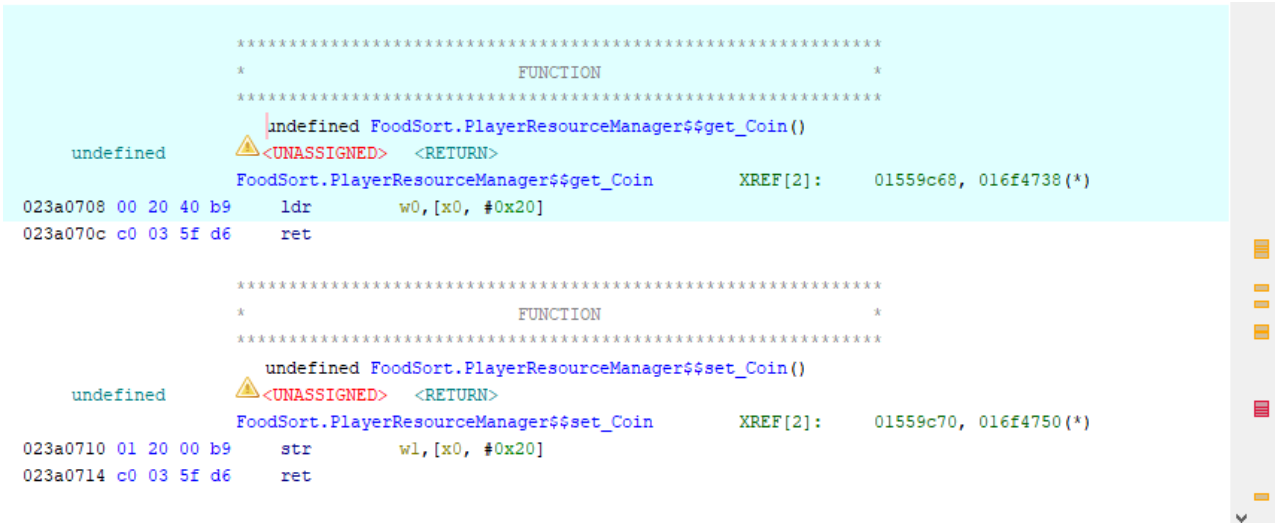
Chạy lại ghidra.py và chọn script.json của cùng lần dump. Console xuất hiện Script finished! và ghidra.py> Finished!. Sau đó lưu project; tên phương thức FoodSort xuất hiện trong Symbol Tree.

```
ghidra.py> Running...
Script finished!
ghidra.py> Finished!
```

Hình 17. Thông báo hoàn tất của lần chạy script thành công

6 Xác định nơi đọc và ghi field coin trong mã native

get_Coin tại 0x23A0708 chỉ gồm lệnh ldr w0, [x0, #0x20] rồi ret. Lệnh đọc giá trị 32 bit từ offset 0x20 của đối tượng và trả về; khớp private int _coin trong dnSpy. set_Coin tại 0x23A0710 thực hiện str w1, [x0, #0x20] rồi ret, ghi giá trị truyền vào cùng field.



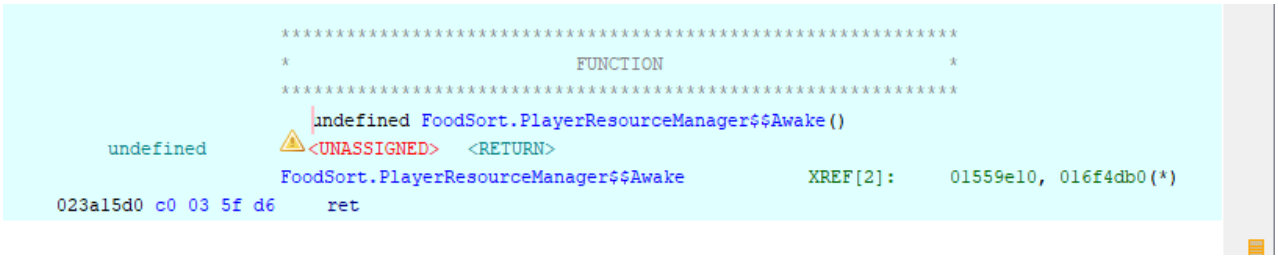
Hình 18. Listing của get_Coin và set_Coin xác nhận đọc và ghi offset 0x20

Kết quả này xác định vị trí field trong đối tượng PlayerResourceManager. Offset 0x20 không phải số coin, RVA hoặc địa chỉ cố định của một đối tượng khi game chạy. Kiểu undefined4 trong Decompiler chỉ phản ánh việc Ghidra chưa khôi phục tên kiểu cho dữ liệu 4 byte; dnSpy cho biết field là int.

Hai hàm accessor không chứa phép cộng hoặc lời gọi PlayerPrefs trong đoạn đã xem. Vì vậy, chúng là điểm đối chiếu field, không được kết luận là nơi tính thưởng hoặc nạp dữ liệu ban đầu. Việc nạp coin được tìm trong Start; phép thay đổi số dư được tìm trong StateCoin.

6.1 Loại trừ Awake khỏi luồng nạp coin

Awake tại 0x23A15D0 chỉ có lệnh ret. Đây là bằng chứng native để loại phương thức này khỏi ứng viên trực tiếp đọc hoặc khởi tạo coin trong bản thư viện đang phân tích.



Hình 19. Awake của PlayerResourceManager chỉ trả về, không nạp coin

6.2 Logic coin khi vào game

Hàm xác định là FoodSort.PlayerResourceManager.Start, RVA 0x22A15D4, địa chỉ Ghidra 0x23A15D4. Ở dòng 18, hàm gọi UnityEngine.PlayerPrefs.GetInt với khóa _StringLiteral_3130 và giá trị mặc định 100. Dòng 19 ghi kết quả vào param_1 + 0x20, chính là field _coin đã đối chiếu.

Decompile: FoodSort.PlayerResourceManager$$Start - (libil2cpp.so)

```
1
2  /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
3
4  void FoodSort.PlayerResourceManager$$Start(long param_1)
5
6  {
7    undefined4 uVar1;
8    long lVar2;
9
10   if ((bRam00000000051c39a7 & 1) == 0) {
11     FUN_021f8840(&FoodSort.PlayerResourceManager_TypeInfo);
12     FUN_021f8840(&StringLiteral_4353);
13     FUN_021f8840(&StringLiteral_9554);
14     FUN_021f8840(&StringLiteral_3130);
15     FUN_021f8840(&StringLiteral_7786);
16     bRam00000000051c39a7 = 1;
17   }
18   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_3130,100,0);
19   *(undefined4 *)(param_1 + 0x20) = uVar1;
20   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_9554,3,0);
21   *(undefined4 *)(param_1 + 0x24) = uVar1;
22   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_4353,1,0);
23   *(undefined4 *)(param_1 + 0x28) = uVar1;
24   uVar1 = UnityEngine.PlayerPrefs$$GetInt(_StringLiteral_7786,3,0);
25   *(undefined4 *)(param_1 + 0x2c) = uVar1;
```

Hình 20. Start đọc coin từ PlayerPrefs rồi gán vào field _coin

Luồng giá trị là dữ liệu đã lưu → GetInt → _coin. Nếu khóa tồn tại, số coin được lấy từ lần lưu trước; nếu chưa có khóa, giá trị mặc định là 100. Do đó, không kết luận mọi lần vào game đều nhận lại 100 coin. Các lời gọi GetInt khác ghi vào offset 0x24, 0x28 và 0x2C thuộc các field booster, không gộp chúng vào biến coin.

6.3 Đối chiếu khóa lưu và kết luận luồng khởi tạo

Đi theo _StringLiteral_3130 trong Listing cho thấy chú thích Coin_SandSort ở nhãn StringLiteral_3130. Đây là khóa được truyền vào GetInt trong Start và SetInt trong StateCoin. Việc hai phương thức dùng cùng khóa nối kết bước nạp và bước lưu số dư.



Hình 21. Nhãn StringLiteral_3130 mang nội dung Coin_SandSort

Mã diễn giải tương đương cho phần nạp coin được viết như sau. Đây là cách diễn đạt hành vi đã quan sát, không phải mã nguồn được khôi phục nguyên bản:

```
_coin = PlayerPrefs.GetInt("Coin_SandSort", 100);
```

Kết luận mục 6: Start thực sự nạp hoặc khởi tạo coin; nguồn dữ liệu là PlayerPrefs phía client; giá trị mặc định là 100; đích ghi là field int _coin, offset 0x20. get_Coin/set_Coin xác nhận nơi đọc và ghi field. Awake được loại trừ vì mã native chỉ trả về.

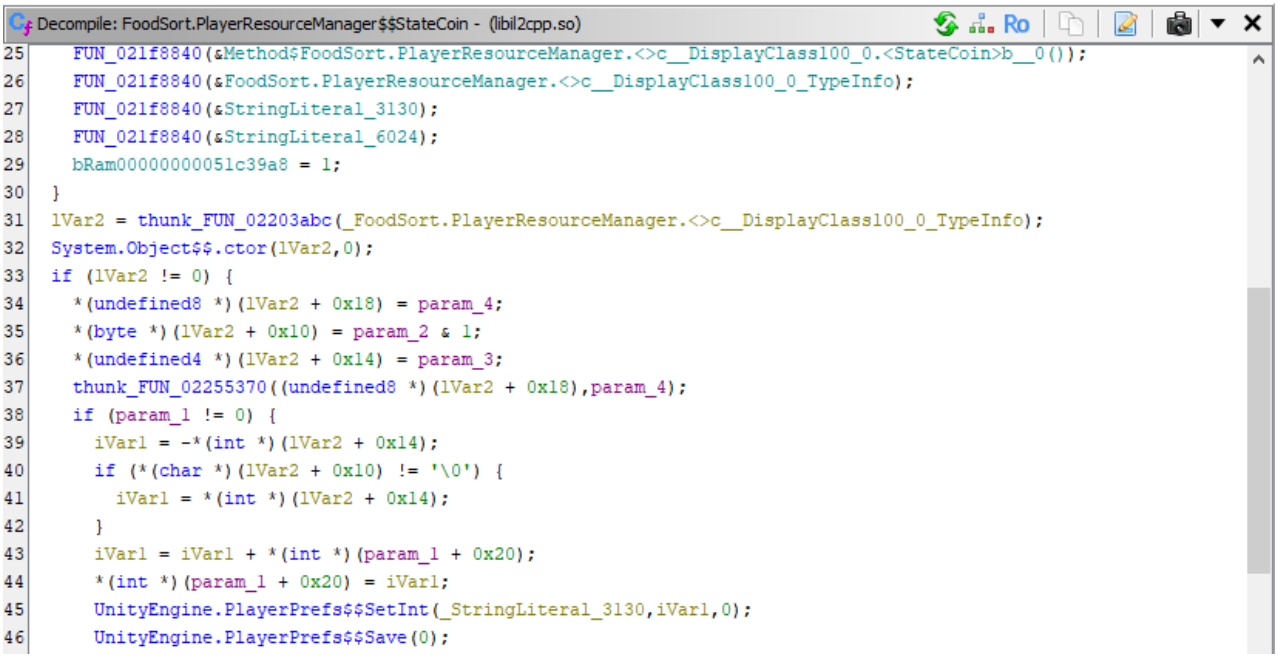
7 Logic cập nhật coin sau khi nhận thưởng

Phần tiếp theo phân biệt nơi thực hiện cộng/trừ số dư với nơi quyết định phần thưởng. PlayerResourceManager.StateCoin nhận lượng coin cần thay đổi; nhánh trong UIWinGame.NextLevel quyết định truyền giá trị 100 và chọn hướng cộng. Phân tích chỉ kết luận nhánh đã quan sát, không áp dụng mặc định cho tất cả nguồn thưởng trong game.

7.1 Hàm StateCoin cộng trừ và lưu số dư

FoodSort.PlayerResourceManager.StateCoin có RVA 0x22A16F8, địa chỉ Ghidra 0x23A16F8. Đối chiếu khai báo dnSpy: param_1 là đối tượng; param_2 tương ứng bool state; param_3 tương ứng int coin. Trong ảnh, tham số state và coin được lưu vào đối tượng phụ trước khi sử dụng, nhưng field số dư vẫn nằm ở param_1 + 0x20.

Dòng 39 khởi tạo số thay đổi bằng -coin. Dòng 40–41 chuyển thành +coin nếu state khác 0. Dòng 43 cộng số thay đổi vào coin hiện tại; dòng 44 ghi lại _coin. Dòng 45 gọi PlayerPrefs.SetInt với khóa _StringLiteral_3130; dòng 46 gọi PlayerPrefs.Save.



Hình 22. Nhánh cộng trừ, cập nhật field và lưu coin trong StateCoin

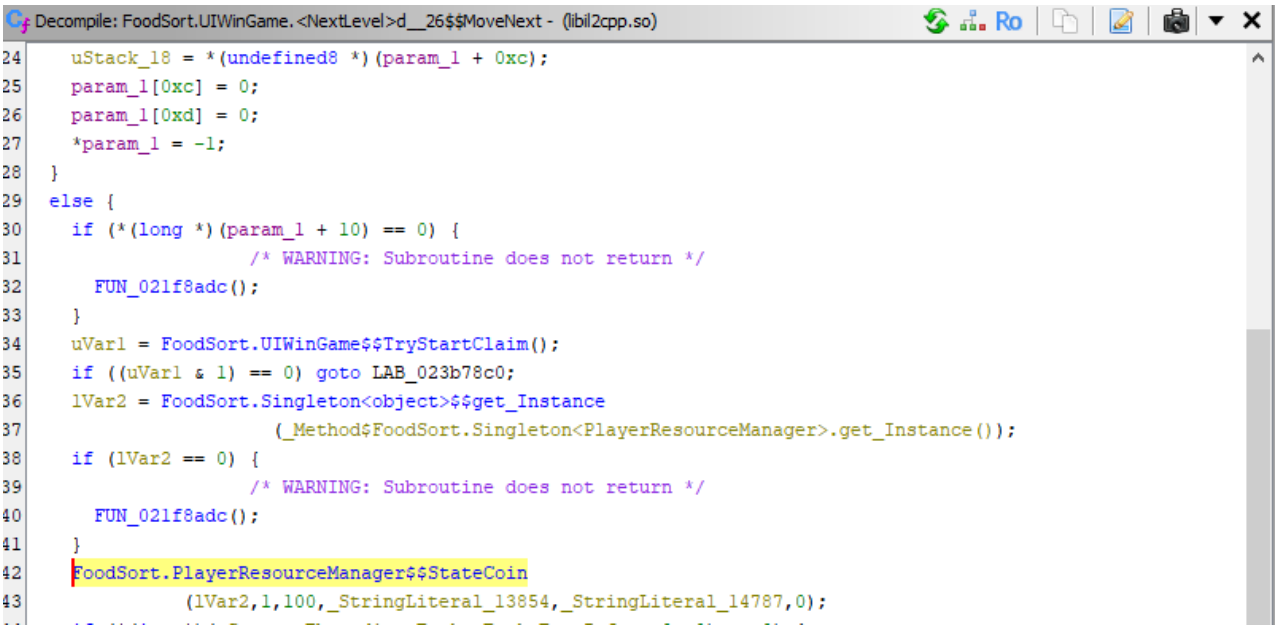
```
int change = state ? coin : -coin;
_coin = _coin + change;
PlayerPrefs.SetInt("Coin_SandSort", _coin);
PlayerPrefs.Save();
```

Với coin là số dương, state = true làm tăng số dư và state = false làm giảm số dư. Phần mã tiếp theo có RecordLevelCoinChange và TrackWhenMetadataReady, nhưng chưa phân tích đầy đủ cơ chế thống kê; không dùng tên những hàm này để suy ra máy chủ xác thực số dư.

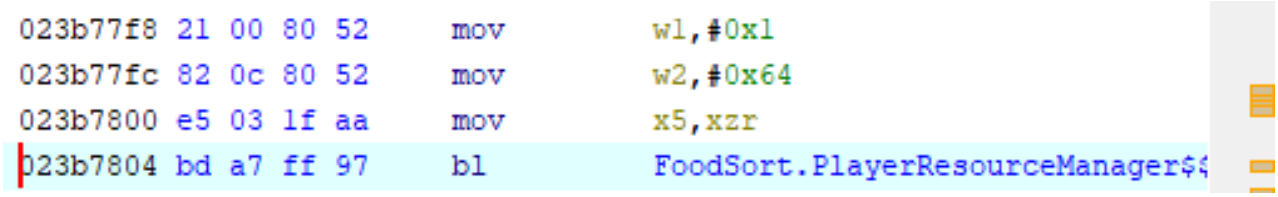
7.2 Nhánh thưởng 100 coin trong luồng NextLevel

UIWinGame.NextLevel tại 0x23B6044, RVA 0x22B6044, chuẩn bị state machine <NextLevel>d__26 và gọi AsyncVoidMethodBuilder.Start. Đoạn nghiệp vụ được quan sát trong <NextLevel>d__26.MoveNext. Trong nhánh này, TryStartClaim phải trả về true trước khi lấy đối tượng PlayerResourceManager và gọi StateCoin.

Dòng 42–43 của ảnh dưới truyền state = 1 và coin = 100. Listing xác nhận mov w1, #0x1 và mov w2, #0x64; 0x64 tương đương 100. Địa chỉ lệnh gọi là 0x23B7804, không được nhầm với địa chỉ bắt đầu NextLevel hoặc MoveNext.



Hình 23. MoveNext kiểm tra TryStartClaim rồi gọi StateCoin với lượng thưởng 100



Hình 24. Lệnh ARM64 thiết lập tham số và gọi StateCoin tại 0x23B7804

Hai tham số chuỗi ở lời gọi mang nhãn _StringLiteral_13854 và _StringLiteral_14787; chưa xác định nội dung nên không tự đặt tên nguồn thưởng. Giá trị 100 ở đây là lượng cộng, khác với 100 mặc định khi GetInt chưa tìm thấy khóa trong Start.

7.3 Điều kiện bắt đầu nhận thưởng

FoodSort.UIWinGame.TryStartClaim có địa chỉ Ghidra 0x23B632C, RVA 0x22B632C. Hàm đọc cờ ở offset 0xA8 vào cVar1. Nếu giá trị ban đầu bằng 0, nó đặt cờ thành 1 và thực hiện các thao tác chuẩn bị. Cuối hàm trả về cVar1 == 0, dựa trên giá trị cờ ban đầu.

Decompile: FoodSort.UIWinGame$$TryStartClaim - (libil2cpp.so)

```
11
12  if ((bRam00000000051c3a90 & 1) == 0) {
13    FUN_021f8840(&UnityEngine.Object_TypeInfo);
14    bRam00000000051c3a90 = 1;
15  }
16  cVar1 = *(char *)(param_1 + 0xa8);
17  if (cVar1 == '\0') {
18    *(undefined2 *)(param_1 + 0xa8) = 1;
19    if (*(long *)(param_1 + 0x50) == 0) {
20LAB_023b63d4:
21                    /* WARNING: Subroutine does not return */
22      FUN_021f8adc();
23    }
24    puVar2 = (undefined8 *)(*(long *)(param_1 + 0x50) + 0x20);
25    *puVar2 = 0;
26    thunk_FUN_02255370(puVar2,0);
27    uVar4 = *(undefined8 *)(param_1 + 0x58);
28    if (*(int *)(_UnityEngine.Object_TypeInfo + 0xe0) == 0) {
29      thunk_FUN_021e5a80();
30    }
31    uVar3 = UnityEngine.Object$$op_Inequality(uVar4,0,0);
32    if ((uVar3 & 1) != 0) {
33      if (*(long *)(param_1 + 0x58) == 0) goto LAB_023b63d4;
34      puVar2 = (undefined8 *)(*(long *)(param_1 + 0x58) + 0x20);
35      *puVar2 = 0;
36      thunk_FUN_02255370(puVar2,0);
37    }
38  }
39  return cVar1 == '\0';
40}
41
```

Hình 25. TryStartClaim kiểm tra cờ nhận thưởng và trả về kết quả điều kiện

Như vậy, cờ ban đầu false cho phép nhánh cộng coin đi tiếp; cờ đang true làm nhánh này bị bỏ qua. Kết luận áp dụng trong trường hợp hàm hoàn tất bình thường; các nhánh gặp đối tượng null có thể đi vào xử lý lỗi. Hai lần gọi liên tiếp khi cờ chưa được reset không đều nhận thưởng.

7.4 Đối chiếu tên cờ và nơi đặt lại

Trong dnSpy, field private bool _isClaimingReward có FieldOffset 0xA8, khớp vị trí TryStartClaim đã đọc. Field _isRewardAdCallbackHandled ở offset 0xA9 là một cờ khác; không gộp hai cờ thành một điều kiện.

```
215         // Token: 0x04000BDD RID: 3037
216         [Token(Token = "0x4000BDD")]
217         [FieldOffset(Offset = "0xA8")]
218         private bool _isClaimingReward;
219
220         // Token: 0x04000BDE RID: 3038
221         [Token(Token = "0x4000BDE")]
222         [FieldOffset(Offset = "0xA9")]
223         private bool _isRewardAdCallbackHandled;
```

Hình 26. Field _isClaimingReward ở offset 0xA8 trong UIWinGame

UIWinGame.OnEnable có RVA 0x22B5B40, địa chỉ Ghidra 0x23B5B40. Trong nhánh UIManager tồn tại, dòng 71 ghi 0 vào offset 0xA8 và dòng 72 gọi BindClaimActions. Điều này cho thấy cờ nhận thưởng được đặt lại khi đi qua nhánh bật giao diện này. Không suy ra cờ chỉ được reset một lần cho mỗi level hoặc đây là toàn bộ cơ chế chống nhận lặp.

```
67      lVar1 = FoodSort.Singleton<object>$$get_Instance
68                        (_Method$FoodSort.Singleton<UIManager>.get_Instance());
69      if (lVar1 != 0) {
70        FoodSort.UIManager$$ShowCoin(lVar1,0);
71        *(undefined1 *)(param_1 + 0xa8) = 0;
72        FoodSort.UIWinGame$$BindClaimActions(param_1);
```

Hình 27. OnEnable đặt lại cờ _isClaimingReward và gọi BindClaimActions

7.5 Tổng hợp luồng và giới hạn chứng minh

Chuỗi xử lý đã xác định là UIWinGame.NextLevel → phần thực thi <NextLevel>d__26.MoveNext → TryStartClaim → PlayerResourceManager.StateCoin(true, 100, ...) → cập nhật _coin → PlayerPrefs.SetInt("Coin_SandSort", số dư mới) → PlayerPrefs.Save. OnEnable cung cấp bằng chứng đặt lại cờ _isClaimingReward trong nhánh đã quan sát.

NextLevel không chứa trực tiếp phép cộng vì đây là hàm bất đồng bộ. Việc truy vết sang MoveNext giúp tìm đúng mã nghiệp vụ thay vì kết luận dựa vào phần tạo AsyncVoidMethodBuilder.

Vị trí	Địa chỉ Ghidra	Ý nghĩa
NextLevel	0x23B6044	Khởi chạy luồng bất đồng bộ
TryStartClaim	0x23B632C	Kiểm tra cờ bắt đầu nhận thưởng
Lời gọi StateCoin	0x23B7804	Truyền state = true và coin = 100
StateCoin	0x23A16F8	Cộng/trừ và lưu số dư
OnEnable	0x23B5B40	Đặt lại cờ trong nhánh đã xem

Chưa xác định đầy đủ sự kiện báo thắng hoặc nút cụ thể nào gọi NextLevel. XREF của NextLevel trong ảnh hiện cho thấy tham chiếu dữ liệu, chưa chứng minh lời gọi trực tiếp từ sự kiện hoàn thành level. Do đó, kết luận là tìm được nhánh thưởng trong lớp giao diện thắng UIWinGame, không khẳng định mọi lần thắng đều tự động cộng ngay 100 coin.

Các ứng viên đã loại trừ hoặc thu hẹp vai trò

Ứng viên	Kết quả đối chiếu và lý do
PlayerResourceManager.Awake	Chỉ có ret tại 0x23A15D0; loại khỏi nơi trực tiếp nạp hoặc cộng coin.
get_Coin và set_Coin	Chỉ đọc/ghi _coin ở +0x20; không có phép cộng hay lưu PlayerPrefs trong đoạn đã xem.
UIWinGame._coinGain	Có kiểu Transform; là thành phần giao diện, không phải field int chứa số dư.
PlayerResourceManager.Start	Đọc số dư đã lưu hoặc dùng mặc định 100; không phải bằng chứng cộng thưởng sau thắng.
UIWinGame.NextLevel	Giữ lại làm đầu mối nhận thưởng; phần thân chỉ khởi chạy state machine, cần đọc MoveNext.

8 Nhận xét an toàn và giới hạn phân tích tĩnh

8.1 Kịch bản can thiệp tài nguyên phía client

Một kịch bản có thể xảy ra là thay đổi dữ liệu lưu cục bộ của khóa Coin_SandSort trước khi game nạp lại. Nếu thay đổi có hiệu lực và không bị cơ chế khác kiểm tra hoặc ghi đè, Start sẽ đọc số đã thay đổi rồi đưa vào _coin. Kịch bản thứ hai là can thiệp lượng thưởng truyền vào StateCoin hoặc thao tác cập nhật số dư, từ đó làm tăng tài nguyên ngoài mức mà nhánh thưởng thông thường quyết định.

Nhận định có cơ sở từ ba bằng chứng: nguồn đọc là PlayerPrefs; phép cộng diễn ra trong libil2cpp.so phía client; số dư mới được ghi lại cục bộ. Trong bài chỉ xác định khả năng và vị trí liên quan, chưa sửa APK, sửa dữ liệu lưu, chạy bản can thiệp hoặc đo tác động trong game.

8.2 Tác động và giới hạn của kết luận

Tác động dự kiến là số dư và phần hiển thị tài nguyên phía client có thể sai lệch, ảnh hưởng các thao tác tiêu coin hoặc cân bằng game nếu chúng tin vào số dư này. Tuy nhiên, các ảnh không chứng minh rằng server chấp nhận số dư sai, không có xác thực từ xa, hoặc người chơi có thể dùng số coin đó cho mọi chức năng.

Phân tích tĩnh chưa xác minh trạng thái lúc chạy, dữ liệu PlayerPrefs thực tế, kết quả hiển thị sau thắng, nội dung hai chuỗi mô tả nguồn thưởng hoặc toàn bộ các nhánh thưởng khác. Các hàm thống kê xuất hiện trong StateCoin cũng chưa được khảo sát đầy đủ. Vì vậy, không kết luận đã khai thác thành công hoặc game hoàn toàn không có cơ chế bảo vệ.

8.3 Cách giảm phụ thuộc vào dữ liệu do client quyết định

Với game có tài khoản và máy chủ, nên để máy chủ quản lý số dư có giá trị, kiểm tra điều kiện hoàn thành level và xác định lượng thưởng hợp lệ. Mỗi giao dịch thưởng cần mã nhận diện để tránh xử lý lặp; client gửi yêu cầu và nhận số dư được xác nhận thay vì tự quyết định phần thưởng. Dữ liệu local có thể dùng làm bộ nhớ đệm để hiển thị, sau đó đối chiếu với trạng thái được máy chủ xác nhận.

Trong chế độ offline, kiểm tra tính toàn vẹn và đối chiếu dữ liệu cục bộ có thể tăng khó khăn cho việc sửa trực tiếp, nhưng vẫn không tạo được mức tin cậy tương đương một nguồn trạng thái độc lập với client. Cờ _isClaimingReward hữu ích để hạn chế thao tác nhận lặp trong luồng giao diện, không phải bằng chứng xác thực số dư từ máy chủ.

9 Kết luận và kết quả cần nộp

Bài thực hành đã nhận diện dữ liệu Unity trong APK, chọn Texture2D bottle 1 trong sharedassets0.assets, xem Preview và xuất tài nguyên. Il2CppDumper tạo dữ liệu tra cứu, dnSpy cung cấp cấu trúc lớp và offset, còn Ghidra xác nhận hành vi native. Việc bật Jython và chỉ định runtime giúp ghidra.py gắn tên thành công trên Ghidra 12.1.2.

Nội dung	Kết quả xác định
Nạp coin khi vào game	PlayerResourceManager.Start, RVA 0x22A15D4, địa chỉ 0x23A15D4; đọc Coin_SandSort, mặc định 100, ghi _coin +0x20.
Cập nhật số dư	StateCoin, RVA 0x22A16F8, địa chỉ 0x23A16F8; cộng/trừ theo state, ghi field rồi lưu PlayerPrefs.
Nhánh thưởng đã xem	NextLevel.MoveNext gọi StateCoin(true, 100, ...) tại 0x23B7804 khi TryStartClaim cho phép.
Điều kiện nhận thưởng	Cờ _isClaimingReward tại +0xA8; được kiểm tra và đặt lại trong các nhánh đã phân tích.
Phần chưa xác định	Đường gọi từ sự kiện hoàn thành level/nút cụ thể đến NextLevel; kiểm chứng động và vai trò server.

9.1 Thư mục kết quả

Kết quả kèm báo cáo gồm file asset đã xuất trong Asset_xuat; dump.cs, DummyDll, script.json và ghidra.py đã thêm runtime Jython; project Ghidra đã lưu và ảnh minh chứng. Gói SandSort_14-9.zip cùng thư mục APK_giai_nen được giữ để đối chiếu theo hướng dẫn GVTH. Khi nộp, ghi tên gói đầu vào đã xác nhận ở mục 1.1 và mã nhận diện thư viện ở mục 1.4; bổ sung thông tin phiên bản ứng dụng nếu xác minh được từ APK.

9.2 Tài liệu sử dụng

1. Tài liệu thực hành của giảng viên: Thực hành dịch ngược, bài Kỹ thuật dịch ngược game Android, các phần B, C và D.

2. Il2CppDumper: mã script ghidra.py và chức năng các đầu ra tra cứu. https://github.com/Perfare/Il2CppDumper

3. Ghidra: hướng dẫn cài extension và lịch sử thay đổi cơ chế chọn runtime Python. https://github.com/NationalSecurityAgency/ghidra